

Actualización (06-sep): el modelo de entrega cambió — todo va con **push a tu rama entregas_apellido_nombre** (ya existe, te la creamos) y **SIN pull request** hasta el final del semestre. Donde esta guía diga «abre tu pull request», ignóralo. La rutina vigente está en el [GIT-CHEATSHEET](#).

MPOO 2027-1 · GRUPO 6 · GUÍA PERMANENTE

Git y GitHub: la rutina de **cada práctica**

De la P1 a la P12 los pasos son exactamente los mismos. Deja esta página en tus marcadores: es la que vas a abrir cada miércoles hasta noviembre. La explicación larga y desde cero está en el [Laboratorio 0](#).

TU SISTEMA

Windows

macOS

Linux

Cómo funciona la entrega en este curso

Hay **un solo repositorio** para todo el grupo y tú eres **colaborador**: trabajas directo en él, sin copias ni forks. La rama `main` está **protegida** (nadie puede escribirle directamente, ni tú ni por accidente), así que cada práctica viaja así:

```
main (protegida, aquí vive lo aceptado)
|
|→ creas tu rama      p03-ramirez
|   trabajas, guardas commits ahí
|   la subes con push
|   abres un pull request ← esa es tu entrega
|
|← yo lo reviso y lo integro a main
```

Así entregan los equipos profesionales, y es lo que vas a poner en tu currículum: un semestre entero trabajando con ramas y pull requests en un repositorio compartido.

Solo la primera vez

Esto ya lo hiciste en el Laboratorio 0. Está aquí por si estrenas máquina o algo se perdió:

1. **Preséntate con Git** (con tu nombre real y el correo de tu cuenta de GitHub):

```
git config --global user.name "Ana Ramírez"
git config --global user.email "tu-correo@ejemplo.com"
```

2. **Sé colaborador.** Me mandas tu usuario (`usuario: fulanito123`) y te llega una invitación al correo de tu cuenta: acéptala. Sin esto, GitHub te va a rechazar el push.
3. **Clona el repositorio del curso:**

```
mkdir -p ~/mpoo && cd ~/mpoo
git clone https://github.com/OscarRuiz21/mpoo-2027-1.git
cd mpoo-2027-1
```

La rutina, completa

La vas a repetir doce veces este semestre. Tómale captura o deja esta página guardada.

Rutina de entrega

De la práctica 1 a la 12 · siempre igual

0 Antes de empezar: trae lo nuevo

```
cd ~/mpoo/mpoo-2027-1
```

```
git checkout main
```

```
git pull
```

Baja la guía y el código de apoyo que subí esta semana. **Cinco segundos y te evita trabajar con la guía vieja.**

1 ``` git checkout -b pNN-apellido ```

Tu rama de la práctica. Nunca trabajas en `main` : está protegida y no te va a dejar.
Ejemplo: `p03-ramirez` .

2 Trabaja

Tu código va en `entregas/apellido_nombre/pNN/` . Nunca toques la carpeta de otro ni `guias/` ni `apoyo/` .

3 `git status`

¿Qué cambió? Míralo siempre antes de guardar. Si aparece un archivo que no esperabas, ahí lo cachas.

4 `git add entregas/apellido_nombre/pNN`

Preparo lo que quiero guardar. Nombra tu carpeta, no uses `git add .` : eso agarra todo lo que haya, incluida basura del IDE.

5 `git commit -m "P03: arreglos y arreglos multidimensionales"`

Lo guardo con un mensaje que explique qué hice. Empieza con el número de práctica. Un mensaje como "cambios" o "asdf" no le sirve a nadie, empezando por ti en noviembre.

6 `git push -u origin pNN-apellido`

Subo mi rama a GitHub. El `-u origin ...` solo la primera vez que subes esa rama; después basta `git push` . Sin push no hay nada.

7 Abre tu PR y comprueba

En el repo del curso: **Compare & pull request** → título `PNN · Apellido` → **Create**.
Tu entrega es el PR, y la hora que cuenta es la de tu último push.

Dónde va tu código

Tu carpeta usa **tu primer apellido y tu primer nombre, en minúsculas y sin acentos**, unidos por guion bajo, y adentro una carpeta por práctica:

mpoo-2027-1/

```
├─ guías/      ← material que yo subo: se lee, no se toca
├─ apoyo/      ← código de apoyo: igual
└─ entregas/
    └─ ramirez_ana/      ← apellido_nombre, la tuya
        ├─ p01/
        │   └─ HolaMundo.java
        ├─ p02/
        │   └─ ...
        └─ p03/
```

El pull request, en corto

- **Título:** `PNN · Apellido`, por ejemplo `P03 · Ramírez`. En la descripción, una línea de qué hiciste.
- **La hora que cuenta** es la de tu **último push** a la rama, no la de abrir el PR.
- **¿Te faltó un archivo?** No abras otro PR: guarda, `git add`, `commit` y `push` en la misma rama. El PR se actualiza solo con todo lo nuevo.
- **Después:** yo lo reviso y lo integro a `main`. GitHub borra tu rama al integrarla y eso es normal: la próxima práctica empieza con el paso 0 y una rama nueva.

La contraseña que no es contraseña

La primera vez que haces `push` en una máquina, la terminal pide usuario y contraseña, pero **la contraseña de GitHub ya no funciona ahí**: se usa un **token** que generas una sola vez, así:

1. GitHub → tu foto arriba a la derecha → **Settings**.
2. Hasta abajo: **Developer settings** → **Personal access tokens** → **Tokens (classic)**.
3. **Generate new token (classic)**. Nómbralo `mpoo`, marca la casilla **repo**, y généralo.
4. **Cópialo ya**: no lo vuelves a ver. Pégallo donde te pide la contraseña.

Guarda el token

Pégalo en tus notas. Lo vas a necesitar cada vez que subas algo desde una máquina nueva, y los laboratorios de la Facultad se borran al reiniciar. Si lo pierdes, no pasa nada: generas otro con los mismos pasos.

Los errores de siempre

LO QUE PASA	QUÉ SIGNIFICA Y QUÉ HACES
<code>nothing to commit</code> después del <code>add</code>	No guardaste el archivo en el editor, o lo pusiste en otra carpeta. Corre <code>git status</code> y revisa la ruta.
Hiciste <code>commit</code> pero no aparece en GitHub	Falta el <code>push</code> . El commit guarda en tu computadora; el push lo sube. Es el error número uno.
<code>protected branch</code> al hacer <code>push</code>	Commiteaste en <code>main</code> sin querer. No pasa nada: corre <code>git checkout -b pNN-apellido</code> ; la rama nueva se lleva tus commits, y desde ahí pusheas normal.
Olvidaste el <code>-u origin ...</code> en el primer push	Git te lo dice con el comando exacto en pantalla. Cópialo y córrelo: es eso.
Te pide contraseña y la rechaza	La contraseña de GitHub no sirve para <code>push</code> : usa tu token (sección de arriba). Si lo perdiste, genera otro.
GitHub rechaza tu push con <code>403</code> o "permission denied"	Casi seguro no has aceptado la invitación de colaborador. Revisa el correo de tu cuenta de GitHub (y el spam), o entra a <code>github.com/OscarRuiz21/mpoo-2027-1/invitations</code> .
<code>Your branch is behind 'origin/main'</code>	Estás parado en <code>main</code> y te falta lo nuevo: <code>git pull</code> . Es exactamente el paso 0.

Si te atorras y no sabes qué hiciste

No borres la carpeta ni vuelvas a clonar. Corre `git status`, tómale captura y mándamela. Casi siempre son dos comandos para salir, y aprender a leer `git status` vale más que cualquier atajo.

Mini glosario

PALABRA	QUÉ ES
repositorio	La carpeta del proyecto con toda su historia de cambios adentro.
clonar	Bajar el repositorio completo a tu máquina, historia incluida. Se hace una vez.
rama	Una línea de trabajo aparte: lo que hagas ahí no toca lo de nadie hasta que se integra.
commit	Una fotografía de tus cambios, guardada en tu máquina con un mensaje que la explica.
push	Subir tus commits a GitHub. Hasta el push, nadie más los ve.
pull	Traer a tu máquina lo nuevo que hay en GitHub.
pull request (PR)	"Aquí está mi rama, revísenla e intégrenla." En este curso, tu entrega.
merge	Integrar una rama en otra. Lo hago yo al aceptar tu PR, y tu trabajo pasa a <code>main</code> .

MPOO 2027-1 · Grupo 6 · Facultad de Ingeniería, UNAM

Prof. Oscar Manuel Ruiz Hurtado · Guía permanente de entrega · agosto de 2026